Задания на лабораторные работы по курсу СПО 2026

Лабораторная работа №1. Взаимодействие программ: события, запуск процессов, IPC и мониторинг

Требуется написать три программы, которые работают в связке:

- Программа A – консольная программа, которая рисует на экране простое изображение (например, счётчик, строку текста или псевдографику, по варианту) и меняет его при нажатии клавиш. Программа принимает начальные параметры из командной строки, пишет лог своих событий в файл и умеет принимать команды от других процессов.
- Программа B — программа с простым интерфейсом (консольным или графическим), в поля которой вводятся параметры для программы A. По нажатию кнопки она запускает программу A с этими параметрами. По нажатию другой кнопки — отправляет программе A сообщение, которое меняет её состояние.
- Программа C — программа, которая следит за работой программы A: определяет, какая клавиша была нажата и когда, пишет время запуска и завершения программы A в отдельный лог. По нажатию выбранной клавиши программа C отправляет программе A дополнительную команду.

Ход работы:

1. Программа A не должна завершаться после первой команды. Она работает в цикле: ждёт событие (нажатие клавиши или команду от другого процесса), обрабатывает его, обновляет экран, снова ждёт. Это и есть цикл событий — основа любой интерактивной программы.
    a. В Linux используйте termios для перевода терминала в неканонический режим и read для чтения одного символа. Для вывода используйте ANSI-escape-последовательности: \033[31m для цвета, \033[2J для очистки экрана, \033[H для перемещения курсора.
    b. В Windows требуется создавать окно через Win32 API, можно сделать оконное приложение с обработкой WM_KEYDOWN в оконной процедуре — это ближе к исходной лабораторной №1.
2. Программа A должна реагировать на нажатие клавиши на клавиатуре. Выберите минимум четыре клавиши, которые меняют отображение. Конкретный набор зависит от вашего варианта.

3. Программа A должна принимать данные из командной строки. Параметры задают начальное состояние. Если параметр не передан, используется значение по умолчанию. Разбор аргументов напишите сами через argc/argv, не используйте готовые библиотеки.
4. Программа A при каждом событии дописывает в файл program_a.log строку вида: 2025-01-15 14:32:11 key=+ size=22. В конце работы допишите строку 2025-01-15 14:35:02 exit.
5. Программа A должна иметь возможность получения данных другой программы:
    a. В Linux создайте именованный канал (FIFO) командой mkfifo /tmp/program_a_fifo и откройте его на чтение в неблокирующем режиме. Команды, записанные в FIFO другой программой, должны обрабатываться так же, как нажатия клавиш.
    b. В Windows используйте оконные сообщения (WM_COPYDATA, PostMessage).
6. Программа B должна иметь поля для ввода параметров программы A: цвет, размер, текст, а также поле для команды, которую нужно отправить. В консольном варианте — просто последовательный ввод с подсказками. В графическом варианте — окно с полями и кнопками. Для графики в Linux можно использовать GTK или Qt, в Windows — Win32 API или тот же GTK/Qt.
7. Программа B должна запускать программу A и передавать данные через параметры консоли:
    a. В Linux используйте fork + execvp или posix_spawn. Соберите массив аргументов из введённых полей.
    b. В Windows используйте CreateProcess с командной строкой, собранной из полей.
    c. Убедитесь, что программа A действительно получает параметры и начинает работу с нужным состоянием.
8. Программа B должна передавать введнные в форму данные в программу A:
    a. В Linux откройте FIFO на запись и запишите туда команду, например key=+ или cmd=color:blue. Программа A должна прочитать эту команду и обработать её так же, как нажатие клавиши.
    b. В Windows используйте PostMessage/SendMessage с WM_COPYDATA.

9. Обработка ошибок. Что если программа A не запустилась? Что если FIFO не существует? Что если программа A уже завершилась? Программа B должна печатать понятное сообщение и не падать.
10. Программа C должна убедиться, что работает именно программа A, а не другая. Для этого:
    a. В Linux проверьте, что процесс с таким именем существует (/proc/<pid>/comm), и что его командная строка соответствует ожидаемой (/proc/<pid>/cmdline).
    b. В Windows используйте EnumProcesses и GetModuleBaseName, или проверьте заголовок окна через FindWindow.
11. Программа C следит поведением и передачей информации между программами A и B: какая клавиша, какое время. Дополнительно программа C пишет свой лог program_c.log, в который добавляет время запуска и завершения программы A.
12. Добавить возможность «иньекции» клавиши в программу
    a. В Linux при появлении выбранной клавиши (например, +) программа C записывает в FIFO дополнительную команду (например, key=t — смена цвета текста). То есть нажатие одной клавиши в программе A приводит к дополнительному действию.
    b. В Windows используйте AutoIt: напишите скрипт, который проверяет, что активное окно — это программа A, читает нажатия клавиш, пишет лог и отправляет дополнительную клавишу через Send. AutoIt — это отдельный язык, он не собирается через CMake; положите скрипт рядом и опишите в README, как его запускать.
13. Программа C должна корректно переживать ситуации: программа A ещё не запущена, программа A уже завершилась, лог пуст, FIFO недоступен. В каждом случае — понятное сообщение и продолжение работы.
14. Сценарий демонстрации. Запускается программа C, затем программа B, из неё запускается программа A с параметрами, программа B отправляет команду, программа C логирует и инжектирует, программа A завершается, программа C фиксирует завершение.
15. Убедитесь, что сборка работает в Linux и Windows. Для Linux используйте CMake и, при необходимости, Makefile. Для Windows — CMake с генератором Visual Studio или MinGW. Для AutoIt-скрипта сделайте отдельный README-раздел.

16. Реализуйте в Linux мониторинг через /proc/<pid>/stat и /proc/<pid>/fd, а в Windows — через SetWindowsHookEx для перехвата клавиатурных сообщений. Это не обязательно, но даёт лучшее понимание того, как ОС предоставляет информацию о процессах. Не используйте ptrace без необходимости — это сложно и требует прав.

В результате у вас должны быть: program_a.c, program_b.c, program_c.c (или .cpp), program_c.au3 для Windows, CMakeLists.txt, README.md, логи program_a.log и program_c.log после демонстрации. Программа A должна реагировать на клавиши и команды, программа B — запускать A и отправлять сообщения, программа C — читать лог, писать свой лог и инжектировать команды.

В отчёте приведите: задание; код всех трёх программ; результат работы программы B с командной строкой (параметры, переданные в A); результат работы программы B с передачей сообщений; результат работы программы C (лог с временем нажатий, временем запуска и завершения, инжектированной командой); объяснение, чем отличается работа с процессами, каналами и событиями в Linux и Windows; описание того, как вы проверяли, что мониторите именно программу A.

Варианты:

1. Изменение размера отображаемых символов
2. Калькулятор сложение
3. Изменение цвета фона
4. Изменение цвета символов
5. Изменение шрифта символа
6. Вывод числа в квадрате
7. Калькулятор умножение
8. Калькулятор вычитание

Лабораторная работа №2. Сборка программы из модулей и статическая библиотека

Вам нужно написать консольную программу «Калькулятор» на языке C, разбитую на несколько модулей, собрать её вручную и через CMake, а затем изучить получившиеся артефакты. Программа должна работать и в Linux, и в Windows.

1. Создайте три файла: calc.h, calc.c и main.c. В calc.h объявите функции ядра: сложение, вычитание, умножение, деление, а также функцию, которая по символу операции (+, -, *, /) выбирает нужную и возвращает результат. В calc.c реализуйте эти функции. В main.c напишите точку входа: программа читает аргументы командной строки вида calc 2 + 3 или строки со стандартного ввода, вызывает ядро и печатает результат. Если пользователь ввёл что-то неверное (не число, неизвестная операция, деление на ноль), программа должна напечатать понятное сообщение об ошибке и завершиться с ненулевым кодом возврата. Обратите внимание: ядро не должно знать о том, как программа общается с пользователем, — это разделение ответственности между модулями.
2. Соберите программу вручную в Linux. Посмотрите на объектные файлы и библиотеку командами nm calc.o, nm libcalc.a, objdump -t calc.o, readelf -s libcalc.a, ar t libcalc.a. Найдите в выводе символы add, sub, mul, div и main. Напиши подробный отчет по выводу и работе линкера.
3. Напишите CMakeLists.txt, в котором есть статическая библиотека calc и исполняемый файл calc_app, линкующийся с ней. Соберите проект в отдельной папке build. Убедитесь, что в Linux появляется libcalc.a, а в Windows — calc.lib (или libcalc.a при сборке через MinGW). Запустите программу. Сравните, какие команды выполнил CMake внутри: посмотрите на файлы в папке build, на CMakeCache.txt, на link.txt.
4. Напишите простой Makefile для Linux с целями all, clean и правилами для .o, .a и исполняемого файла. Добейтесь, чтобы make собирал всё с нуля, а make clean удалял артефакты.
5. Если вы работаете в Windows, соберите проект через Visual Studio или MinGW-w64. Для просмотра символов используйте dumpbin /symbols calc.obj, dumpbin /headers calc.exe, lib /list calc.lib. Сравните с выводом Linux-инструментов: какие секции есть, как называются символы, как оформляются имена функций.
6. Создайте отдельный исполняемый файл test_calc, который проверяет работу ядра на 10–15 случаях: обычные числа, отрицательные, ноль, деление на ноль, большие числа. Тесты должны печатать «OK» или «FAIL» по каждому случаю и возвращать ненулевой код, если хотя бы один тест провалился. Подключите тесты к CMake через add_test и enable_testing, чтобы их можно было запускать командой ctest. Убедитесь, что тесты не зависят от main.c — они линкуются только с ядром.

7. Попробуйте специально вызвать ошибку: уберите определение функции mul из calc.c и соберите проект. Посмотрите, какое сообщение выдаст линкер. Затем верните функцию и убедитесь, что всё снова собирается.

В результате у вас должны быть: исходные файлы calc.h, calc.c, main.c, test_calc.c; файл CMakeLists.txt; файл Makefile для Linux; папка build с собранными артефактами; README с командами сборки и запуска для Linux и Windows. Программа должна корректно работать с аргументами командной строки и со стандартным вводом, обрабатывать ошибки и возвращать корректные коды возврата. Тесты должны проходить через ctest.

В отчёте приведите: задание; команды сборки вручную и их вывод; вывод nm, objdump, readelf, ar (Linux) и dumpbin, lib (Windows); объяснение, чем отличаются .o, .a/.lib и исполняемый файл; объяснение, что такое символ, релокация и таблица символов; сравнение Makefile и CMake; результаты ctest; пример ошибки линковки и её объяснение.

Варианты:

1. Калькулятор. Ядро: add, sub, mul, div, выбор операции по символу. CLI: calc 2 + 3. Тесты: 15 случаев, включая деление на ноль.
2. Статистика массива. Ядро: min, max, sum, avg, median. CLI: stats 3 1 4 1 5. Тесты: пустой массив, один элемент, отрицательные числа.
3. Перевод чисел между системами счисления. Ядро: to_base(n, base), from_base(str, base), проверка корректности цифр. CLI: convert 255 16 → FF. Тесты: основание 2, 8, 10, 16, некорректные символы.
4. Проверка строк. Ядро: is_palindrome, is_anagram, count_vowels, to_upper. CLI: strcheck palindrome "А роза упала на лапу Азора". Тесты: пустая строка, пробелы, регистр, не-ASCII.
5. Комплексные числа. Ядро: структура complex, c_add, c_sub, c_mul, c_abs. CLI: complex 3 4 → модуль 5. Тесты: ноль, чисто мнимые, большие значения.
6. Матричные операции. Ядро: mat_add, mat_mul, mat_transpose, mat_det (2×2, 3×3). CLI: matrix add с вводом размеров. Тесты: единичная матрица, нулевая, несовместимые размеры.
7. Простой шифр. Ядро: caesar_encode, caesar_decode, xor_cipher. CLI: cipher caesar 3 "Hello". Тесты: пустая строка, сдвиг 0, сдвиг 33(26), не-буквы.
8. Работа с датами. Ядро: is_leap_year, days_in_month, day_of_week, days_between. CLI: date weekday 2025 1 15. Тесты: високосные годы, границы месяцев, переход через год.

Лабораторная работа №3. Динамическая библиотека, плагины и сетевая загрузка

Вам нужно превратить программу из второй работы в приложение с плагинами. Ядро программы не должно знать, какие операции существуют: оно загружает их из динамических библиотек во время работы. Дополнительно нужно реализовать загрузку плагина по сети, параллельную загрузку нескольких плагинов и, по желанию, получение информации об устройстве через системный интерфейс.

1. Создайте заголовочный файл plugin.h, в котором объявлены две функции: const char* plugin_name(void) и int plugin_op(int a, int b).
2. Создайте отдельные исходные файлы add.c, sub.c, mul.c, div.c. Каждый из них реализует plugin_name и plugin_op. Соберите каждый плагин как отдельную динамическую библиотеку: в Linux — .so, в Windows — .dll. В Linux используйте add_library(add MODULE add.c), в Windows не забудьте про __declspec(dllexport) для экспортируемых функций. Проверьте, что каждая библиотека содержит ровно те символы, которые нужны, и не тянет за собой лишние зависимости.
3. Требуется реализовать ядро calc_app с загрузкой плагинов, которое при старте сканирует папку plugins/, находит все .so/.dll, загружает их через dlopen/LoadLibrary, получает адреса функций через dlsym/GetProcAddress и складывает в реестр плагинов. Если плагин не загрузился, не экспортирует нужные функции или падает — ядро печатает ошибку и продолжает работу. Реализуйте CLI: calc_app list печатает список плагинов, calc_app add 2 3 вызывает плагин add, calc_app all 2 3 вызывает все плагины по очереди. Обратите внимание: ядро не должно содержать имён конкретных операций в коде — только работу с реестром.
4. Исследование зависимостей
    a. В Linux посмотрите на плагины командами ldd, readelf -d, nm -D.
    b. В Windows — dumpbin /dependents, dumpbin /exports.
    c. Посмотрите, какие зависимости у ваших плагинов: тянут ли они libc, есть ли у них неразрешённые символы. Попробуйте специально удалить один плагин — убедитесь, что ядро продолжает работать.
5. Добавление загрузка плагина по сети. Напишите два приложения: plugin_server и plugin_client. Сервер по запросу отдаёт содержимое .so/.dll файла. Клиент скачивает файл во временную папку и загружает его как плагин. Используйте обычные TCP-сокеты: в

Linux — POSIX-сокеты, в Windows — Winsock (WSAStartup, socket, connect, send, recv). Обработайте ошибки: сервер недоступен, файл повреждён, плагин не загрузился. Обратите внимание: скачанный плагин нужно сохранить с правильным расширением, иначе dlopen/LoadLibrary его не примет.

6. Параллельная загрузка плагинов через команду calc_app load-all, которая загружает все плагины из папки параллельно. Используйте потоки: pthread в Linux, CreateThread в Windows. Реестр плагинов защитите мьютексом (pthread_mutex_t или CRITICAL_SECTION). Сравните время параллельной и последовательной загрузки на 20–50 плагинах, приведите замеры в отчёте.
7. Реализуйте команду calc_app reload, которая выгружает все плагины и загружает их заново. При выгрузке убедитесь, что ни один поток не использует плагин: для этого достаточно остановить рабочие потоки через мьютекс и флаг. Попробуйте специально вызвать dlclose во время работы с плагином и посмотрите, что произойдёт.
8. Реализуйте плагин device_info, который через ioctl в Linux (/dev/...) или DeviceIoControl в Windows получает информацию об устройстве и возвращает её как строку. Если у вас нет подходящего устройства, используйте /dev/null или любое другое безопасное устройство.

В результате у вас должны быть: заголовок plugin.h; четыре и более плагина; ядро calc_app; сервер plugin_server и клиент plugin_client; README с командами сборки и запуска; отчёт с выводом ldd/dumpbin, замерами параллельной загрузки, описанием ABI, PLT/GOT и работы с сокетами в двух ОС. Программа должна корректно работать, когда часть плагинов отсутствует или повреждена.

Опишите: что такое C-ABI и почему для плагинов удобно использовать именно C, а не C++; как ядро находит и загружает плагины; как работает hot-reload и почему он требует синхронизации; как устроены сокеты в Linux и Windows и в чём различия; что такое ioctl и DeviceIoControl и зачем они нужны. Приведите вывод всех использованных утилит и замеры.

Лабораторная работа №4. Низкоуровневый файловый ввод-вывод и системные вызовы

Вам нужно написать две утилиты: mycopy для копирования файлов и myinfo для вывода информации о файле. Дополнительно нужно реализовать многопоточное копирование и передачу файла по сети.

1. Напишите утилиту на низкоуровневом API. В Linux используйте open, read, write, close, в Windows — CreateFile, ReadFile, WriteFile, CloseHandle. Программа принимает два аргумента: исходный и целевой файл. Поддерживает опции: --buf N для размера буфера, --progress для вывода прогресса, --checksum для подсчёта простой контрольной суммы (например, XOR всех байт). Файлы открывайте в бинарном режиме: в Windows для этого нужно _setmode(_fileno(stdout), _O_BINARY), если вы печатаете бинарные данные. Обработайте все ошибки: файл не найден, нет прав, диск переполнен, неверные аргументы. В Linux печатайте errno и strerror(errno), в Windows — GetLastError() и FormatMessage. Сделайте версию без буферизации и с буферизацией.
2. Напишите вторую версию через stdio. Используйте fopen, fread, fwrite, fclose. Сравните скорость работы на файлах разного размера (1 КБ, 1 МБ, 100 МБ) и при разных размерах буфера (512, 4096, 65536, 1 МБ). В Linux прогоните обе версии под strace -c и сравните количество системных вызовов read/write. В Windows можно использовать Process Monitor. Приведите таблицу с результатами и объясните, почему stdio буферизует ввод-вывод и как это влияет на число системных вызовов.
3. Напишите myinfo, которая выводит: размер файла, тип (обычный файл, каталог, символическая ссылка, другое), права в Linux (rwx) или атрибуты в Windows (readonly, hidden, system), время последнего изменения. Используйте stat/lstat в Linux и GetFileAttributesEx в Windows.
4. Реализуйте опцию многопоточной обработки в утилите --threads N. Файл разбивается на N диапазонов, каждый поток копирует свой диапазон. В Linux используйте pthread_create и pwrite, в Windows — CreateThread и WriteFile с OVERLAPPED или просто SetFilePointerEx + WriteFile. Убедитесь, что итоговый файл совпадает с исходным (сравните контрольные суммы). Замерьте время для 1, 2, 4, 8 потоков на файле 100 МБ и приведите результаты. Попробуйте запустить на SSD и на HDD (если есть) и сравните.
5. Реализуйте опцию myinfo --raw, которая через ioctl в Linux или DeviceIoControl в Windows получает дополнительные сведения об устройстве, на котором лежит файл. Например, в Linux можно вызвать HDIO_GET_IDENTITY для жёсткого диска или BLKGETSIZE64 для размера блочного устройства. В Windows — IOCTL_DISK_GET_DRIVE_GEOMETRY. Если у вас нет прав или

подходящего устройства, опишите, какие коды вы пробовали и что получили в ответ.

В результате у вас должны быть: mycopy.c, mycopy_stdio.c, myinfo.c, fileserver.c; CMakeLists.txt с целями для Linux и Windows; README с командами; отчёт с таблицами замеров, выводом strace -c, сравнением stdio и низкоуровневого API, описанием многопоточного копирования и сетевой передачи.

Опишите: что такое файловый дескриптор и хендл и чем они отличаются; почему stdio буферизует; как работают pwrite/OVERLAPPED; почему многопоточное копирование ускоряет работу не всегда; как устроены сокеты в Linux и Windows; что такое ioctl и DeviceIoControl. Приведите все замеры и вывод утилит.

Варианты:

1. mycopy. Копирование файла и информация о файле. Опции: --buf, --progress, --checksum, --threads. myinfo показывает размер, тип, права, время.
2. mycat + mywc. mycat выводит файл на экран, mywc считает строки, слова, символы, байты. Опции: --lines, --words, --bytes, --all. Сравнить с системными cat/wc.
3. mygrep. Поиск подстроки или простого шаблона в файле. Опции: --ignore-case, --line-numbers, --count, --invert. Поддержать поиск в нескольких файлах. Сравнить с системным grep.
4. myhexdump. Вывод файла в шестнадцатеричном виде с ASCII-колонкой. Опции: --width N, --offset, --limit. Сравнить с hexdump -C и xxd.
5. mytail + myhead. Вывод первых или последних N строк файла. Опции: --lines N, --bytes N, --follow (для tail). Для --follow использовать чтение с отслеживанием позиции.
6. mydiff. Сравнение двух файлов построчно. Вывод: номера различающихся строк, содержимое обеих версий. Опции: --ignore-case, --ignore-whitespace, --brief. Сравнить с системным diff.
7. mysort. Сортировка строк файла. Опции: --numeric, --reverse, --unique, --output FILE. Реализовать свою сортировку (например, слиянием) и сравнить с qsort из libc.
8. mybase64. Кодирование и декодирование Base64. Опции: encode, decode, --input FILE, --output FILE. Реализовать алгоритм самостоятельно, проверить на бинарных файлах.

Лабораторная работа №5. Память, отладка и утечки

Вам нужно реализовать динамический массив (vector) и простой пул-аллокатор и структуру данных (из варианта), написать тесты, найти и исправить ошибки с помощью санитайзеров и отладчиков, а также сделать пул потокобезопасным.

1. Создайте файлы структур данных, например vector.h и vector.c. Объявите структуру vector с полями data, size, capacity. Реализуйте функции vector_init, vector_push, vector_get, vector_free. Внутри используйте malloc, realloc, free. Функции должны возвращать коды ошибок: не удалось выделить память, выход за границы, неверные аргументы. При vector_push, если size == capacity, увеличьте вместимость вдвое. Напишите тест, который добавляет 10 000 элементов и проверяет, что все они на месте. Обратите внимание: realloc может переместить блок памяти, поэтому все указатели на элементы становятся недействительными после вызова.
2. Создайте pool.h и pool.c. Объявите функции pool_init(size_t block_size, size_t count), pool_alloc(), pool_free(void* p), pool_destroy(). Внутри пул выделяет один большой блок через malloc, разбивает его на count блоков по block_size и хранит список свободных блоков. При pool_free блок возвращается в список. Реализуйте защиту от двойного освобождения: помечайте блоки флагом или используйте отдельный список занятых. Напишите тест: 1 000 выделений и освобождений, проверка, что память не течёт и не пересекается.
3. Искусственно добавьте в код три ошибки: утечку памяти (забыли free), выход за границы массива (запись в data[size]) и использование памяти после освобождения (use-after-free). Прогоните тесты под санитайзерами: в Linux — gcc -fsanitize=address,undefined -g, valgrind --leak-check=full, gdb; в Windows — Visual Studio Debugger, ASan (если доступен), Dr. Memory. Приведите в отчёте вывод санитайзера до и после исправления и объясните, как вы нашли каждую ошибку.
4. Сделайте вторую версию пула pool_mmap, которая вместо malloc использует mmap в Linux или VirtualAlloc в Windows. Сравните скорость и потребление памяти с первой версией на 10 000 и 100 000 блоков. Подробно опишите сравнение и выводы.
5. Добавьте в пул мьютекс: pthread_mutex_t в Linux, CRITICAL_SECTION в Windows. Реализуйте pool_alloc_ts и pool_free_ts, которые захватывают мьютекс. Напишите тест: 8 потоков одновременно выделяют и освобождают по 10 000

блоков. Прогоните тест под ThreadSanitizer (-fsanitize=thread в Linux) и убедитесь, что гонок нет. Замерьте время работы с мьютексом и без него (однопоточный вариант) и приведите результаты.

6. Замерьте скорость malloc/free из libc на тех же сценариях и сравните с вашим пулом.

В результате у вас должны быть: vector.h, vector.c, pool.h, pool.c, pool_mmap.c (или общая реализация с #ifdef), тесты, CMakeLists.txt, README. Все тесты должны проходить, санитайзеры не должны находить ошибок после исправления, пул должен быть потокобезопасным.

Опишите: как устроен malloc/free и чем он отличается от пула; как работает mmap/VirtualAlloc; как находить утечки и ошибки с помощью санитайзеров и valgrind/Dr. Memory; как сделать аллокатор потокобезопасным и какие подводные камни есть у мьютекса. Приведите вывод всех инструментов до и после исправления, таблицы замеров и сравнение с libc.

Варианты

1. Vector + pool. Динамический массив и пул фиксированных блоков. Тесты: 10 000 добавлений, 1 000 выделений/освобождений. Санитайзеры, mmap-версия, потокобезопасность.
2. String builder. Динамическая строка с операциями append, append_char, insert, remove, substring. Внутри realloc, при переполнении — удвоение. Тесты: длинные строки, вставка в середину, удаление.
3. Связный список на пуле. Односвязный и двусвязный список, узлы выделяются из пула, а не через malloc. Операции: push_front, push_back, remove, find, reverse. Тесты: 10 000 узлов, удаление из середины.
4. Arena allocator. Арена-аллокатор: одна большая область, из которой выделяются блоки последовательно, освобождение — только всей ареной сразу. Операции: arena_init, arena_alloc, arena_reset, arena_destroy. Сравнить с malloc по скорости.
5. Ring buffer. Кольцевой буфер фиксированного размера. Операции: rb_init, rb_push, rb_pop, rb_peek, rb_is_full, rb_is_empty. Тесты: переполнение, опустошение, обёртка индексов.
6. Hash table с пулом. Хеш-таблица с открытой адресацией или цепочками, узлы из пула. Операции: ht_insert, ht_get, ht_remove, ht_free. Тесты: 10 000 вставок, коллизии, удаление.
7. Бинарное дерево на пуле. Двоичное дерево поиска, узлы из пула. Операции: tree_insert, tree_find, tree_remove, tree_traverse (in-order,

pre-order, post-order). Тесты: сбалансированные и вырожденные деревья.

8. Обработка файла через mmap. Программа отображает файл в память через mmap/CreateFileMapping, обрабатывает его (например, считает слова или заменяет символы) и записывает результат. Сравнить с чтением через read. Тесты: большие файлы, пустые файлы, файлы только для чтения.